ASTER: Authorization-Scoped Threshold Exact-Domain Credential Derivation with Failure-Safe Root-Epoch Healing

Yuanyi Zhang
Hangzhou Information Technology Branch, Information Technology Institute, China Railway Shanghai Group Co., Ltd., Hangzhou 310009, Zhejiang, China
E-mail: revanton@icloud.com

Abstract

Enterprise networks continue to contain applications, appliances, directories, operation consoles, and vendor-maintained systems whose only modifiable authentication secret is a policy-constrained text password. Deterministic password generators avoid storing service-password values, but most practical constructions place a reusable master or lineage key on the endpoint during derivation. A compromise that coincides with legitimate use can therefore expand from one observed password to an entire credential lineage or, under a global root, to multiple services. Moving the root to a distributed PRF service removes direct key reconstruction but introduces a distinct deployment risk: an authorization that is not bound to the complete credential context can be amplified into many legitimate outputs while the distributed key remains hidden. Moreover, proactive share refresh protects an unchanged secret against mobile compromise but does not heal a root key that has already been disclosed.

This paper presents ASTER, an authorization-scoped threshold architecture for exact-domain legacy credential derivation and post-compromise Root-Epoch healing. ASTER compiles each bounded password policy into a finite accepted language and uses inverse Rank/Unrank functions. A threshold evaluator realizes a keyed permutation over the exact rank domain, so generation g deterministically maps to one accepted password without replacement. Each evaluation requires a short-lived, single-use capability bound to the complete credential context, Root-Epoch, password generation, operation, freshness generation, expiry, and nonce. Normal operation releases only the authorized password; neither a Root-Epoch key nor a reusable per-lineage derivation key is reconstructed on the endpoint. When root compromise is suspected, ASTER generates an independent new Root-Epoch key and migrates credentials individually. Candidate passwords are checked against authenticated cross-epoch history inside the distributed computation, and a durable evidence-bounded state machine preserves both old and candidate reconstruction paths when the remote password-change outcome is ambiguous.

We formalize exact-domain injectivity, capability confinement, authorization-budget non-amplification, failure-safe migration, safe epoch retirement, and scoped post-compromise healing. In the artifact, 121 exactly translated policies compiled successfully; 97 policy domains completed 9.7 million derivations with no policy violation, duplicate, replay mismatch, or Rank/Unrank failure, while 24 oversized domains failed closed. Exact capabilities produced no authorization spill in a 32-context universe, whereas deliberately projected and wildcard capabilities admitted concrete spill. Independent Root-Epoch replacement reduced old-root exposure from 100 credentials to zero as migrations committed; share refresh alone changed no sampled output. Ninety-six injected adapter traces and a 777-state TLA+ model preserved the migration invariants, and all eight broken models produced counterexamples. Finally, three- and five-party malicious honest-majority MP-SPDZ executions agreed with an independent fixed vector, but their single-host loopback medians of 33.01 s and 125.53 s show that the generic circuit is feasibility evidence rather than a deployment-ready backend. ASTER does not claim a new MPC, secret-sharing, OPRF, FPE, or finite-language primitive; its contribution is the credential-specific composition that simultaneously removes reusable derivation secrets from the normal endpoint, constrains output authority, and supports failure-safe recovery after complete old-root disclosure.

Keywords: deterministic credential derivation; threshold cryptography; password policy; format-preserving encryption; capability authorization; post-compromise security; password rotation; legacy authentication

1. Introduction

Passwords are no longer the preferred authentication mechanism for systems that can adopt phishing-resistant public-key authentication, passkeys, or WebAuthn. Nevertheless, large organizations often operate a long tail of legacy systems whose verifier cannot be replaced in the short term. Typical examples include vendor portals, network appliances, operation consoles, database gateways, directory-bound applications, and internally developed systems whose only supported interface is a conventional password field. In this setting, credential management is a migration problem: the organization wants stronger governance without requiring immediate modification of every relying party.

A natural response is deterministic password derivation. PwdHash demonstrated domain-separated password transformation without server changes [1]. PALPAS derived service passwords from a high-entropy secret and non-secret per-service salts while synchronizing only metadata [2]. AutoPass specified a password generator designed around site-specific rules and password changes [3]. These systems show that a legacy verifier can be supplied with a strong textual password without storing every service password as an encrypted vault entry.

The security boundary, however, is often still concentrated at the client. If the endpoint reconstructs a global Root Key or a reusable per-lineage key during legitimate use, an attacker that compromises the process at the right time can learn more than the one password the user intended to reveal. This is qualitatively different from observing one derived password. The derived password is an output for one service and one generation; a reusable derivation key is authority over a larger output set.

Threshold cryptography provides an obvious tool for removing the long-term secret from one machine. NIST describes the threshold paradigm as secret-sharing a key across multiple parties and executing the keyed operation through secure multiparty computation so that the key need not be reconstructed [4,5]. Distributed PRF work similarly shows that key derivation can be performed across multiple servers. Pythia introduced a verifiable partially-oblivious PRF service and supported efficient key rotation [6]. LaKey demonstrated low-round distributed PRFs for scalable distributed key derivation and evaluated implementations in MP-SPDZ [7]. RFC 9497 standardizes OPRF, VOPRF, and POPRF interfaces [8]. Baecker et al. further provide a fully adaptive threshold partially-oblivious PRF with proactive key refresh [9].

These developments make it insufficient to claim novelty merely from splitting a root key among several servers. They also expose two problems that are easy to conflate.

First, **key secrecy is not output authorization**. A distributed evaluator can protect its master key while exposing a broad application interface. If a capability intended for one credential can be replayed against multiple service identifiers, account identifiers, or generations, an endpoint compromise can retrieve many legitimate outputs without ever reconstructing the root. The relevant security quantity is therefore not only “how many evaluator domains must be compromised before the root is learned?” but also “how many credential outputs become derivable from the authority available to the compromised endpoint?”

Second, **share refresh is not root-compromise recovery**. Proactive resharing protects the same underlying secret from an adversary that accumulates stale shares over time. Once the root itself has been learned, resharing that same root under new shares cannot make the attacker forget it. Healing requires a new, independent root and a migration protocol that moves relying-party passwords from the old cryptographic epoch to the new one without losing track of which password the remote target actually accepts.

ASTER is designed around these two distinctions. It retains the useful property of exact, deterministic, policy-compliant password reconstruction while removing reusable derivation secrets from the normal endpoint. It then adds explicit Root-Epoch replacement and evidence-bounded migration.

The central design principle is:

> **The endpoint should receive the least reusable authority compatible with the requested legacy login: one authorized password output for one complete credential context and one generation.**

ASTER makes four technical contributions.

1. **Non-reconstructing exact-domain credential sequence.** A bounded password policy is compiled to a finite accepted language L_P. Exact Rank/Unrank functions provide a bijection between L_P and $[0, N_P)$, where $N_P = |L_P|$. A threshold evaluator applies a context-separated keyed permutation over $[0, N_P)$, and generation g is the permutation input. Thus each generation maps deterministically to one accepted password, with strict same-lineage non-repetition until the domain is exhausted, while the normal endpoint never receives the permutation key.
2. **Generation-scoped authorization and non-amplification.** Every evaluation requires a capability bound to the full canonical credential context, Root-Epoch, generation, operation, freshness generation, expiry, nonce, and use budget. We formalize a conditional non-amplification property: below the unrestricted-derivation threshold, q valid single-use exact-scope capabilities authorize at most q distinct requested outputs. Projection-bound and wildcard capabilities are explicit negative controls.
3. **Root-Epoch post-compromise healing.** ASTER distinguishes proactive refresh of an unchanged root from replacement after root disclosure. A new Root-Epoch key is independently generated. Credentials are migrated individually, with cross-epoch password-history exclusion performed inside the distributed computation. A credential becomes healed only after authoritative evidence commits it to the new epoch.
4. **Evidence-bounded failure-safe migration.** Legacy password-change interfaces are not transactions. A timeout can occur before or after the target commits the new password. ASTER durably prepares a candidate descriptor before submission and commits only when adapter-defined evidence identifies the candidate as authoritative. Ambiguous outcomes preserve both old and candidate reconstruction paths. An old Root-Epoch cannot be erased while any committed or unresolved state still depends on it.

The contribution is deliberately at the **credential protocol and systems-security layer**. ASTER does not claim that secret sharing, MPC, OPRFs, FPE, finite-language ranking, or generic capabilities are new. The novelty claim is the joint contract needed by legacy credential management: exact-domain no-repeat derivation, non-reconstructing normal operation, exact output authorization, true root replacement after disclosure, private cross-epoch history exclusion, and failure-safe remote migration.

The remainder of the paper is organized as follows. Section 2 positions ASTER against deterministic generators, password-policy work, format-preserving encryption, threshold cryptography, and distributed PRFs. Section 3 defines the system and threat model. Section 4 defines exact-domain credential sequencing. Section 5 specifies scoped threshold evaluation. Section 6 defines Root-Epoch healing and remote migration. Section 7 states security properties. Section 8 describes the implementation architecture. Section 9 presents the evaluation. Section 10 discusses limitations and deployment guidance, and Section 11 concludes.

2. Related Work and Novelty Boundary

2.1 Deterministic password generation

PwdHash transforms a user password into a site-specific credential in the browser and requires no server modification [1]. Its principal goal is cross-site separation rather than enterprise lifecycle management. PALPAS derives service passwords from a high-entropy secret and per-service salts, storing only salts and related metadata on a synchronization service [2]. AutoPass provides a detailed password-generator design and explicitly considers site rules and forced password changes [3]. These systems establish the feasibility and practical motivation of deterministic legacy-verifier compatibility.

ASTER differs in the authority model. The endpoint does not normally possess the reusable master or per-lineage secret that defines the credential sequence. A legitimate output is produced only after exact-scope authorization and threshold evaluation.

2.2 Password-policy languages and verified generation

Password managers must satisfy heterogeneous composition policies. Gautam, Lalani, and Ruoti designed a password-composition policy description language based on a large collection of real policies and demonstrated libraries and proof-of-concept integrations [10]. Grilo et al. used EasyCrypt to specify and verify a password-generation algorithm and integrated the verified component into a Bitwarden prototype [11]. These works establish that policy semantics and uniform generation are substantial problems in their own right.

ASTER does not claim a richer policy language. It assumes a bounded canonical policy that can be compiled into a finite deterministic transition system. Unsupported semantics fail closed rather than being silently approximated. The research contribution begins after a finite accepted language has been defined: ASTER maps password generations into that exact domain through a distributed permutation and binds each evaluation to an explicit authorization.

2.3 Arbitrary-domain ciphers and format-preserving encryption

Black and Rogaway studied ciphers on arbitrary finite domains [12]. Bellare et al. formalized format-preserving encryption and analyzed rank-then-encipher and cycle-walking approaches for complex formats [13]. NIST SP 800-38G standardizes FF1 and historically FF3; the second public draft of Revision 1 removes FF3 and strengthens the FF1 domain-size requirement in response to cryptanalytic and implementation concerns [14].

Accordingly, ASTER does **not** claim that Rank/Unrank, rank-then-encipher, cycle walking, or FF1 are novel. The exact-domain permutation abstraction is used because an independent hash-and-reduce construction can introduce modulo bias and collisions across generations, while a permutation directly provides an injective sequence. The production threshold backend may use secure MPC around an established exact-domain construction; the choice of backend is an implementation decision whose assumptions must be separately audited.

2.4 Secret sharing, threshold cryptography, and distributed PRFs

Shamir secret sharing provides information-theoretic secrecy below threshold in its ideal model [15]. Modern threshold systems extend this idea from key storage to key usage. NIST's Multi-Party Threshold Cryptography program explicitly describes cryptographic operations performed through MPC while the secret key remains shared [4], and NIST IR 8214C calls for technical specifications, reference implementations, and experimental evaluation of multi-party threshold schemes [5].

Pythia is a verifiable partially-oblivious PRF service that supports efficient bulk key rotation [6]. LaKey reduces the online round complexity of distributed PRF-based key derivation and demonstrates MPC implementations [7]. RFC 9497 standardizes OPRF/VOPRF/POPRF protocol interfaces [8]. The fully adaptive threshold POPRF construction of Baecker et al. adds proactive key refresh and composable security [9].

These results narrow ASTER's novelty boundary. A paper that merely distributes a master secret, refreshes shares, or performs a threshold PRF evaluation would not be sufficient. ASTER instead treats threshold evaluation as a dependency and asks what the application must enforce around it: exact credential context, generation, operation, freshness, bounded authority, and Root-Epoch replacement after complete old-root disclosure.

2.5 Multi-factor key derivation and recovery

MFKDF generalizes password-based key derivation to multiple authentication factors and includes threshold constructions for factor loss [16]. Subsequent analysis identified weaknesses in the original constructions and highlighted the importance of precise threat models and state integrity [17]. This episode is directly relevant to ASTER's methodology: cryptographic building blocks are insufficient if metadata integrity, scope, and lifecycle state are underspecified.

ASTER separates **break-glass recovery** from **normal derivation authority**. Offline recovery material can be used to reconstruct or reconstitute evaluator infrastructure under explicit recovery procedures, but the normal endpoint is not provisioned with a credential that automatically releases enough remote shares to recreate the Root-Epoch key.

2.6 Passwordless authentication

Passkeys and WebAuthn should be preferred when relying parties can migrate to phishing-resistant public-key authentication. ASTER is not a replacement for those mechanisms. It is a transitional control for systems whose verifier still requires a textual password.

2.7 Positioning summary

Table 1 positions ASTER by the specific properties claimed in this paper. “Not specified” does not imply that a cited system cannot be extended; it means the published design does not define the property as part of its contract.

Table 1. Positioning against closest classes of prior work.

Approach	Legacy text password	No stored service-password values	Exact accepted-space sequence	Endpoint avoids reusable derivation key	Per-generation authorization	Root replacement after root disclosure	Failure-safe ambiguous rotation
PwdHash [1]	Yes	Yes	No	No	No	No	No
PALPAS [2]	Yes	Yes	Not specified	No	No	Not specified	Not specified
AutoPass [3]	Yes	Yes	Not specified	No	No	Not specified	Not specified
Pythia [6]	Application dependent	PRF service	No	Yes for PRF key	Partially-oblivious input model	Key rotation, not credential migration	No
LaKey [7]	Application dependent	PRF service	No	Yes	Application dependent	Not the credential-level focus	No
Threshold POPRF [9]	Application dependent	PRF service	No	Yes	POPRF public input	Proactive refresh of same key	No
ASTER	Yes	Yes	Yes	Yes	Yes	Yes	Yes

ASTER’s novelty is not “distributed password generation.” It is the **explicit relation among exact-domain generation, output-scoped authority, Root-Epoch replacement, and ambiguous remote lifecycle state**.

3. System Model, State, and Threat Model

3.1 Entities

ASTER consists of the following entities.

- **Client** C: the user-facing endpoint that requests a password for a legacy target. It stores authenticated non-secret credential metadata and a durable migration journal. It is allowed to see a password that the user is legitimately using, but it should not receive a reusable Root-Epoch or lineage key in normal operation.

- **Approval Authority** A: an independently controlled service or operator domain that issues short-lived capabilities for credential derivation or migration. Its policy may require user presence, device attestation, administrator approval, or another enterprise control.
- **Evaluator domains** $E_1, \ldots, E_n$: independently controlled parties holding shares of a Root-Epoch secret or participating in an equivalent threshold/MPC construction. A qualified threshold jointly evaluates the credential permutation.
- **Legacy target** R: a relying party whose modifiable secret is a textual password. ASTER does not require the target to know ASTER metadata.
- **Freshness service** F (optional but recommended): an independent compare-and-set or append-only checkpoint that records the latest authenticated Root-Epoch/generation digest and helps detect complete rollback of local metadata.
- **Recovery trustees**: offline or administratively independent holders used for break-glass recovery or evaluator reconstitution. They are not a normal evaluation oracle.

3.2 Credential context

For a credential record, ASTER defines a canonical context

$$C = (v, s, a, \ell, \sigma, p, h_p, e_p),$$

where v is a vault identifier, s is a service identifier, a is an account identifier, ℓ is a random lineage identifier, σ is a credential salt, p is the policy identifier, h_p is the canonical policy hash, and e_p is the policy epoch.

Lifecycle metadata additionally contains:

- Root-Epoch e_r;
- committed credential generation g;
- freshness generation f;
- recent authenticated history descriptors (e_r, g) needed by the target's password-history rule;
- adapter metadata;
- an optional pending migration operation.

The service-password value itself is never persisted as a vault entry.

3.3 Root-Epochs

A Root-Epoch e identifies an independently generated threshold secret K_e. Epoch replacement is deliberately stronger than share refresh:

$$K_{e+1} \xleftarrow{\$} K, K_{e+1} \text{ independent of } K_e.$$

Refreshing the shares of K_e can be useful against a mobile adversary, but it does not change the Root-Epoch and does not satisfy ASTER's healing condition after K_e has been disclosed.

3.4 Adversary

The adversary may:

1. read all non-secret credential metadata and policy descriptions;
2. observe any password legitimately released to the endpoint;

3. compromise the endpoint before, during, or after a legitimate derivation;
4. steal or roll back the local metadata database;
5. drop, delay, duplicate, reorder, or replay network traffic;
6. crash the client at any persistence or network boundary;
7. compromise fewer than the threshold number of evaluator domains;
8. obtain stale or already-used authorization material;
9. fully learn an **old** Root-Epoch key K_e in the post-compromise-healing experiment.

We separately discuss compromise of the Approval Authority or a threshold of evaluator domains because either can cross the intended authorization boundary.

3.5 Assumptions

The security claims rely on the following assumptions.

- The threshold backend meets its stated key-secrecy and correctness goals.
- The exact-domain evaluator implements a permutation over the specified rank domain for a fixed key and tweak.
- The Approval Authority's signature or MAC is unforgeable; any prehash used for request binding is collision resistant.
- Capabilities use canonical unambiguous encoding.
- Each honest evaluator independently verifies the complete capability scope, expiry, freshness generation, revocation state, and use budget before participating.
- Durable replay state is not silently rolled back together with the local endpoint, or an external freshness anchor detects such rollback.
- The adapter's commit predicate faithfully represents the evidence source it claims to use; a malicious trusted adapter can defeat classification by lying about its observations.

3.6 Security goals

ASTER targets the following goals.

- **G1 Exact policy compliance.** Every released password belongs to the target's accepted policy language.
- **G2 Deterministic reconstruction.** Fixed authenticated state reconstructs the same credential.
- **G3 Same-lineage non-repetition.** Within one Root-Epoch, context, and policy, distinct generations map to distinct passwords until the finite domain is exhausted.
- **G4 Endpoint non-materialization.** Normal derivation does not place a Root-Epoch key or reusable per-lineage derivation key on the endpoint.
- **G5 Authorization confinement.** A capability for one exact request cannot be redirected to another request.
- **G6 Failure-safe migration.** Remote ambiguity does not destroy the ability to reconstruct both plausible credentials.
- **G7 Root-compromise healing.** Disclosure of K_e does not determine credentials conclusively migrated to independently generated K_{e+1}.
- **G8 Safe retirement.** An epoch is erased only after no committed or unresolved credential depends on it.

Non-goals include protecting a plaintext password that malware legitimately observes at the moment of use, guaranteeing availability against malicious evaluators, making a weak target password policy strong, hiding all service inventory metadata, and providing forward secrecy for passwords already observed under a compromised epoch.

4. Exact-Domain Credential Sequence

4.1 Bounded policy language

Let P be a canonical bounded password policy over alphabet Σ and lengths $[L_{min}, L_{max}]$. The intended compiler supports the common finite constraints required by legacy systems: allowed and forbidden characters, class-count minima and maxima, fixed positions, prefixes and suffixes, first/last-character restrictions, per-character maxima, run limits, and a finite set of forbidden substrings. Unsupported semantics are rejected.

The compiler constructs a deterministic finite transition system

$$A_P = (Q, \Sigma, \delta, q_0, F),$$

with accepted language

$$L_P = \{ w \in \Sigma^* : L_{min} \le |w| \le L_{max}, \delta^*(q_0, w) \in F \}.$$

The compiler enforces explicit state and memory budgets because a product of counters, substring automata, run state, and positional constraints can grow exponentially.

4.2 Exact counting

For target length ℓ, position i, and automaton state q, define the suffix count

$$C_\ell(i,q) = \begin{cases} 1, & i = \ell \wedge q \in F, \\ 0, & i = \ell \wedge q \notin F, \\ \sum_{c \in \Sigma : \delta(q,c)\downarrow} C_\ell(i+1, \delta(q,c)), & i < \ell. \end{cases}$$

All counts use arbitrary-precision integers. The exact domain size is

$$N_P = |L_P| = \sum_{\ell = L_{min}}^{L_{max}} C_\ell(0, q_0),$$

and the policy's exact combinatorial capacity is $H_{eff} = \log_2 N_P$ bits. ASTER reports H_{eff} because uniform generation cannot compensate for a small accepted language.

4.3 Ranking and unranking

Fix a canonical order on lengths and characters. Unrank_P(r) first selects the length interval containing rank r. At each position it examines outgoing characters in canonical order; the number of accepted suffixes following each edge gives the interval width. Rank_P(w) performs the inverse accumulation.

Lemma 1 (Bijection). For any successfully compiled non-empty policy, Rank_P and Unrank_P are mutual inverses between L_P and $[0, N_P)$.

Proof sketch. At every prefix, outgoing edge intervals are disjoint and adjacent, and their widths sum to the suffix count of the current state. Thus every rank selects exactly one outgoing interval and every accepted prefix contributes exactly the sum of earlier interval widths. Induction over the remaining positions gives both inverse identities. ▫

4.4 Context-separated permutation

For Root-Epoch e, credential context C, and policy P, ASTER defines an ideal exact-domain permutation

$$\Pi_{e,C,P}:[0,N_P)\to[0,N_P).$$

Generation is the permutation input:

$$r_g=\Pi_{e,C,P}(g),\ pwd_{e,g}=Unrank_P(r_g).$$

Generation is **not** placed only in a tweak that selects an independent permutation for each g; doing so would remove the injective sequence interpretation.

In the deployed system, the permutation key is derived or represented inside the threshold/MPC backend and is never returned to the endpoint. Canonical domain separation includes protocol version, Root-Epoch, full credential context, policy identity, and policy hash.

4.5 Properties

Theorem 1 (Exact compliance and deterministic reconstruction). If the threshold evaluator returns $r_g\in[0,N_P)$ for the authenticated request, then $pwd_{e,g}\in L_P$. Repeating the same authenticated request and backend state returns the same password.

This follows from Lemma 1 and determinism of the fixed permutation.

Theorem 2 (Same-lineage non-repetition). For fixed (e,C,P) and distinct $g_1,g_2\in[0,N_P)$, successful outputs are distinct.

Proof. $\Pi_{e,C,P}$ is injective and Unrank_P is injective; therefore their composition is injective. ▫

Theorem 3 (Ideal-permutation marginal). If the permutation is sampled uniformly from all permutations on $[0,N_P)$, then for any fixed generation g and any $w\in L_P$, the marginal probability of output w is $1/N_P$.

A concrete backend gives computational indistinguishability under its own assumptions, not unconditional randomness over keys.

5. Authorization-Scoped Threshold Evaluation

5.1 Why threshold secrecy alone is insufficient

A threshold PRF can keep the root secret while still exposing an application-level oracle. Suppose the evaluator accepts a signed ticket that binds only (serviceID, accountID) but ignores policy epoch, credential generation, or lineage identifier. One compromised endpoint may then reuse a single authorization across every generation or across multiple records that share the projected fields. The root remains hidden, yet the output set expands.

ASTER treats this as an authorization-amplification failure rather than a root-key failure.

5.2 Capability format

A capability binds the canonical encoding of

protocolVersion
operation
vaultID
serviceID
accountID
lineageID
credentialSalt

policyID
policyHash
policyEpoch
rootEpoch
generation
freshnessGeneration
expiry
nonce
useBudget

The Approval Authority signs or MACs this complete structure. Every evaluator verifies the same canonical representation independently before participating.

The operation field distinguishes at least:

- derive - release one currently authorized credential;
- rotate-candidate - evaluate a candidate for remote password change;
- history-check - compare candidate outputs against authenticated recent history inside the distributed computation;
- reconcile - re-evaluate a committed/candidate pair for an unresolved operation;
- recovery-admin - a separate break-glass administrative operation not callable through ordinary derivation credentials.

5.3 Replay and freshness

A capability is valid only if:

1. its signature/MAC verifies;
2. every bound field matches the requested operation;
3. the freshness generation equals the evaluator's accepted freshness state;
4. the current time is no later than expiry;
5. the nonce is not revoked;
6. the durable use counter is below useBudget.

A default derivation capability has useBudget=1. A reconciliation operation may intentionally allow a small bounded number of repeated evaluations but remains bound to the same operation and credential descriptors.

5.4 Threshold evaluation interface

ASTER requires an evaluator functionality

DPRP.Eval(e, C, g, N_P, cap) -> r_g

with the following contract:

- the threshold parties jointly realize a permutation over $[0, N_P)$;
- fewer than the configured compromise threshold do not learn a reusable Root-Epoch key;
- the client learns only the authorized output (or the final password after Unrank, depending on implementation);
- every honest evaluator rejects incomplete or mismatched scope;
- the endpoint never receives a Root-Epoch key or reusable per-context permutation key.

One implementation route is actively secure MPC around an established arbitrary-domain/FPE construction, with the exact Rank/Unrank compiler running either outside MPC on public policy metadata or partially inside MPC depending on leakage goals. The artifact instantiates a bounded AES-Feistel/cycle-walk circuit in MP-SPDZ as feasibility evidence; Section 9 reports its exact boundary and cost.

5.5 Capability confinement

Let a request x be the complete canonical tuple authorized by a capability. Let Verify(cap,x) denote honest verification by the evaluator threshold.

Theorem 4 (Capability confinement). Assuming unforgeability of the capability authenticator, canonical encoding, and honest scope validation by the required evaluator threshold, a capability issued for x cannot authorize an evaluation at $x' \neq x$ except with negligible probability.

Proof sketch. For $x' \neq x$, the canonical digest differs unless a collision occurs in the binding hash/encoding. Reusing the original signature/MAC with the modified digest fails verification; producing a valid authenticator for the modified digest is a forgery. Replays at x are separately limited by durable nonce/use-budget state. ▫

5.6 Authorization-budget non-amplification

Let T be the multiset of attacker-observed valid capabilities and let q be the total remaining valid use budget across them. Let $E(T)$ be the set of distinct exact credential requests that can be successfully evaluated without additionally compromising an unrestricted-derivation capability or the evaluator threshold.

Theorem 5 (Exact-scope q-capability non-amplification). Under Theorem 4 and durable single-use accounting, $|E(T)| \leq q$.

This statement is deliberately conditional on the deployment access structure. If one compromised endpoint can automatically obtain new approvals, or if one compromised administrative domain controls both approval and enough evaluator shares, the effective threshold collapses. Likewise, if capability fields are projected or wildcarded, one valid token can authorize multiple exact requests; the negative-control experiments in Section 9 quantify that amplification.

5.7 Endpoint compromise during legitimate use

ASTER cannot hide the password that the endpoint must submit to a legacy target. Malware present during legitimate use can capture that password. The reduction in blast radius is that the endpoint does not also receive a reusable root/lineage key. Under an uncompromised approval/evaluator boundary, further outputs require further authorized evaluations.

This distinction is the main security motivation for moving the exact-domain permutation behind an authorization-scoped threshold interface.

6. Root-Epoch Healing and Failure-Safe Migration

6.1 Why proactive refresh is insufficient after root disclosure

Proactive sharing changes the shares of a fixed secret. This is valuable if the attacker compromises different parties across time but never learns a qualified set from one valid sharing state. However, if the attacker has already recovered K_e, new shares of K_e do not revoke the attacker's copy.

ASTER therefore defines two independent lifecycle operations:

- **Share refresh:** replace shares while keeping Root-Epoch e and K_e unchanged. This addresses a mobile-share adversary and is delegated to the selected threshold backend.

- **Root-Epoch replacement:** independently generate K_{e+1} and migrate relying-party credentials. This is the recovery mechanism after suspected or confirmed root disclosure.

6.2 New Root-Epoch generation

Evaluator domains run distributed key generation or an equivalent protocol to create an independent key:

$$K_{e+1} \xleftarrow{\$} K.$$

The new key is not derived from the old key. The freshness service, if used, commits the transition intent and new epoch identifier before individual credential migration begins.

6.3 Per-credential migration state

For a credential currently committed at (e, g), ASTER persists a migration record before contacting the target:

operationID
old = (e, g)
candidateEpoch = e+1
candidateGeneration = j
candidatePolicyHash
preparedAt
adapterContract
status = Prepared

The candidate generation j is selected inside the threshold service as described below. The candidate password may appear transiently for submission but is never stored as the durable source of truth.

6.4 Cross-epoch history exclusion

Same-epoch injectivity does not prevent a password under the new independent permutation from equaling a password in a previous epoch. A target may reject reuse within a history window h.

Let authenticated recent history descriptors be

$$H = [(e_1, g_1), \ldots, (e_m, g_m)], m \le h.$$

The migration evaluator reconstructs these old outputs **inside the distributed computation** and tests candidate generations $j = 0, 1, \ldots$ under $e+1$ until it finds a password not in the history set. Only the selected candidate need be released for remote submission.

This avoids persisting historical password values. If an old Root-Epoch needed for the configured history window has already been irreversibly destroyed, the client cannot truthfully claim history exclusion for that window and must fail closed or use a target-specific reset mechanism.

Proposition 1 (Bounded history exclusion). If the new exact domain contains at least $|H \cap L_P| + 1$ candidate outputs not yet consumed by the migration search, sequential candidate testing terminates after at most $|H \cap L_P| + 1$ distinct candidates.

The proposition does not predict undocumented server-side history beyond the authenticated window.

6.5 Remote password-change ambiguity

Legacy password-change interfaces rarely provide transactional semantics. Consider two traces:

1. the target commits the new password and the response is lost;
2. the request is lost before the target commits anything.

Both may appear locally as a timeout. Committing in both cases can strand the account in trace 2; rolling back in both cases can lose the accepted new credential in trace 1.

ASTER therefore separates transport success from credential-state evidence.

6.6 Evidence classes

An adapter can use one or more target-specific evidence sources, for example:

- authenticated login with the new credential;
- authenticated login with the old credential;
- a target-exposed password-version value;
- an administrative readback endpoint;
- an independent directory replication state;
- a documented overlap-then-revoke contract.

The canonical evidence classes are:

- **NewOnly** - evidence identifies the candidate as authoritative and the old password as non-authoritative;
- **OldOnly** - evidence identifies the old password as authoritative;
- **Both** - both are accepted or the target intentionally overlaps credentials;
- **Neither** - neither credential can be confirmed;
- **Contradictory** - evidence sources disagree;
- **Unknown** - insufficient evidence, timeout, unsafe verification budget, or unavailable readback.

For an atomic replacement contract, only NewOnly permits local commit. OldOnly aborts the candidate. Both, Neither, Contradictory, and Unknown preserve both reconstruction descriptors in UnknownOutcome. A target with an explicitly documented overlap-then-revoke contract can define a separate state transition for Both.

6.7 Migration state machine

The principal state transitions are:

Current state	Action / evidence	Next state
Committed(e,g)	Persist candidate descriptor	Prepared(old=(e,g), candidate=(e+1,j))
Prepared	Submit candidate to target	Submitted / Verifying
Submitted / Verifying	NewOnly	Committed(e+1,j)
Submitted / Verifying	OldOnly	Committed(e,g); candidate aborted
Submitted / Verifying	Both / Neither / Contradictory / Unknown	UnknownOutcome(old=(e,g), candidate=(e+1,j))
UnknownOutcome	Later adapter-authorized reconciliation	Re-enter evidence classification; never silently clear state

UnknownOutcome is a safety state, not an error that may be silently cleared. Reconciliation later repeats only the adapter-authorized observations needed to classify the remote state.

6.8 Safe old-epoch retirement

An epoch e may be retired only if all three conditions hold:

1. no credential is committed to e;
2. no pending or unknown operation references e as old or candidate state;
3. the configured password-history window no longer requires derivation under e, or an approved alternative history-reset procedure has been completed.

Only then may honest evaluator domains erase their shares and durable encrypted backups of K_e.

6.9 Scoped post-compromise healing

Suppose the adversary fully learns K_e. Any credential still committed to e remains exposed; ASTER does not hide this fact. A credential becomes **healed with respect to old-root disclosure** only after it is conclusively committed to independently generated K_{e+1}.

Theorem 6 (Root-Epoch healing). Assume K_{e+1} is independently generated and the adversary does not compromise its threshold. Knowledge of K_e and all public metadata does not computationally determine ASTER outputs under epoch $e+1$, except through authorized evaluations or attacks on the selected threshold permutation backend.

The theorem is a separation statement between independent keys, not a claim that previously observed passwords become secret again.

7. Security Analysis

7.1 Known password pairs

For a known output $(e, g, pwd_{e,g})$, an attacker can compute its exact rank under the public policy if Rank is public. This yields a known input/output pair for the underlying permutation. Security of unseen outputs is therefore the selected PRP/threshold backend's computational security claim; it is not information-theoretic secrecy.

The design intentionally avoids relying on secrecy of policy metadata or generation counters.

7.2 Metadata theft

Stealing the credential metadata database reveals service inventory, account relationships, salts, policy descriptors, epochs, and generations. This information can enable targeted attacks and rollback attempts, so metadata confidentiality may still be desirable. However, under ASTER's root-secrecy assumptions, metadata theft alone does not reveal the threshold Root-Epoch secret or authorize arbitrary evaluations.

7.3 Endpoint compromise

A compromised endpoint during legitimate use can observe the plaintext password and any authorization material presented to it. Under exact single-use capabilities, the immediate derivation authority is bounded by the remaining capability budget. An attacker that persists on the endpoint may request additional approvals through the legitimate UI; preventing that requires Approval Authority policy such as independent administrator confirmation, user presence on a separate device, or rate/behavior controls. ASTER's cryptography cannot manufacture organizational independence that the deployment does not have.

7.4 Approval Authority compromise

If the Approval Authority is compromised and evaluators accept arbitrary correctly signed capabilities, the attacker can request many outputs while the Root-Epoch key remains hidden. This is a **derivation-authority compromise**, not necessarily a root-key compromise. Deployments can reduce this risk by requiring multi-party approval, hardware-backed signing, or separating routine derivation approval from high-risk migration approval.

7.5 Evaluator compromise

Compromising fewer than threshold evaluator domains should not reveal the Root-Epoch key under the selected threshold backend. Compromising a qualified threshold provides unrestricted derivation capability and crosses ASTER's main cryptographic boundary.

The effective threshold must be measured over **independent compromise domains**, not physical node count. If one administrator, credential, hypervisor, or endpoint can control enough evaluator instances, the nominal threshold is meaningless.

7.6 Capability projection and wildcarding

Removing bound fields creates equivalence classes of requests. For example, a token bound only to (serviceID, accountID) may authorize multiple generations, policy epochs, or lineages. The expected output spill is the size of the equivalence class minus the single request that was intended. Section 9 includes projection/wildcard negative controls because they demonstrate that correct threshold cryptography can still be deployed with an unsafe application interface.

7.7 Replay and rollback

Nonce and use-budget checks prevent ordinary capability replay only if replay state is durable. Rolling back the entire evaluator replay database can resurrect an old token. Likewise, rolling back the client metadata can resurrect an old credential generation. A production deployment should therefore combine authenticated local state with an independent monotonic freshness mechanism: hardware monotonic storage, a compare-and-set service, or an append-only audit checkpoint.

7.8 Root-Epoch compromise and share refresh

Refreshing shares of K_e does not heal a disclosed K_e. This distinction is an explicit invariant in ASTER. Conversely, replacing K_e with K_{e+1} without migrating relying-party passwords also does not heal the credential because the remote target still accepts the old password. Healing is complete only at the conjunction of independent new key generation, remote password migration, authoritative commit evidence, and eventual safe retirement of the old epoch.

7.9 Malicious adapter

The state machine prevents the client from inferring success from an ambiguous transport event. It cannot prevent a trusted adapter from fabricating evidence. Adapter code, TLS roots, directory readback channels, and other evidence sources must therefore be authenticated and treated as part of the trusted computing base.

7.10 Small accepted domains

Exact uniformity cannot repair a weak policy. If N_P is small, exhaustive search may be feasible regardless of ASTER. The policy compiler should report exact H_{eff} and enforce a deployment-specific lower bound. The threshold backend may also have a minimum supported domain size; such backend limits are separate from password-strength policy.

7.11 Denial of service and liveness

A malicious evaluator, Approval Authority, or target can deny service. UnknownOutcome deliberately sacrifices automatic liveness rather than guessing a credential state. A target that provides no safe verification channel may require manual reconciliation.

8. Implementation Architecture

8.1 Prototype layers

The research artifact is organized into five layers.

1. **Policy compiler.** Implements exact finite-language compilation, arbitrary-precision suffix counting, and Rank/Unrank without referring to any unpublished predecessor as a comparison system.
2. **Canonical credential state.** Stores service/account identifiers, lineage, salts, policy hash, Root-Epoch, committed generation, freshness generation, adapter metadata, and migration journal.
3. **Approval service.** Issues signed exact-scope capabilities with expiry, nonce, and use budget.
4. **Threshold exact-domain evaluator.** Provides a process-local semantic implementation for exhaustive protocol testing and a separate MP-SPDZ circuit for cryptographic feasibility. In the latter, each computing party privately inputs two 64-bit words; their XOR forms a 128-bit key only inside MPC. A ten-round AES-based balanced Feistel network permutes a 20-bit superset domain, and four fixed-cap cycle-walk steps select a rank in $[0, 1000003)$. Only a success bit and rank are opened. This is a generic circuit composition, not FF1 and not a new threshold primitive.
5. **Remote adapters.** Implement prepare-submit-verify-reconcile behavior for representative HTTP-form and LDAP-style targets.

8.2 Normal derivation flow

A normal credential derivation proceeds as follows.

1. The client loads authenticated credential metadata and reads (e, g, P, C, f).
2. It requests a derive capability for the exact tuple from the Approval Authority.
3. The client sends the request and capability to the evaluator set.
4. Each evaluator independently validates signature, complete scope, freshness, expiry, revocation, and use budget.
5. The threshold computation evaluates $r_g = \Pi_{e,C,P}(g)$.
6. The final password is obtained as Unrank_P(r_g) and returned to the client.
7. The endpoint displays/submits the password and clears transient buffers on a best-effort basis.

No step returns a Root-Epoch key or reusable lineage key to the endpoint.

8.3 Migration flow

Root-Epoch replacement adds internal history exclusion and a durable journal.

1. Evaluators generate independent Root-Epoch $e + 1$.
2. The client requests a migration capability for one record.
3. The distributed service rederives authenticated recent history under old epochs internally.
4. It searches new-epoch generations until it finds a candidate outside the history window.
5. The client persists the candidate descriptor before submission.
6. The adapter submits the candidate password.

7. Evidence classification drives commit, abort, or UnknownOutcome.
8. After all records leave the old epoch and no history/pending dependency remains, the old epoch is retired.

8.4 Concrete cryptographic backend

The prototype uses MP-SPDZ's mal-shamir-bmr-party.x, an actively secure honest-majority Shamir/BMR backend [18]. The three-party configuration has corruption threshold $t = 1$ and the five-party configuration has $t = 2$; all configured parties participate online in the reported runs. These are corruption thresholds, not claims that an arbitrary subset can complete this particular online execution.

The circuit is intentionally conservative about its claim boundary. AES supplies the round function for a ten-round balanced Feistel permutation; a 128-bit prefix of the canonical request hash is the public context tweak, and the round number is independently separated. Generation 42 and domain $N = 1000003$ form a public fixed vector. Cycle walking executes four iterations regardless of the first valid result and fails closed if none lies in the exact domain. The measured vector accepted in one iteration. A clear reference implementation invokes OpenSSL AES-128-ECB and must agree with the MPC result for both party configurations.

MP-SPDZ commit 9d809599ea6ce627216a389ca7d984fbb75d0cb9 required two build-only compatibility patches in its supplied Bullseye image: replacing unavailable C++20 std::barrier with the equivalent Boost barrier, and raising BMR's compile-time maximum party allocation from three to five. Neither patch changes the ASTER circuit or threshold protocol.

8.5 Semantic reference model

The executable semantic model treats the threshold exact-domain service as an idealized internal component. Its purpose is to validate request scoping, no-repeat behavior, Root-Epoch transitions, unknown outcomes, and retirement guards. It is **not** evidence for threshold-cryptographic security or production latency.

The model includes:

- exact request digest construction;
- signed capability issuance and verification;
- expiry and single-use replay checks;
- a finite exact-domain permutation stand-in;
- deterministic password reconstruction;
- cross-epoch history exclusion;
- Root-Epoch migration state;
- unknown-outcome preservation;
- safe-retirement checks.

Nine semantic tests cover the properties summarized in Section 9.2; the Rust implementation and TLA+ model provide independent implementation-level and abstract-state checks.

9. Evaluation

The evaluation separates policy-space correctness, protocol semantics, threshold-backend feasibility, failure behavior, and public-metadata scalability. Every quantitative statement below is generated from a machine-readable result file; local semantic timings are never used as a substitute for MPC measurements. The artifact records macOS 26.5.2 on x86-64, Rust 1.87.0, Python 3.9.6 for result generation, Java 21 for TLC, Docker 29.3.1, and MP-SPDZ commit 9d809599ea6ce627216a389ca7d984fbb75d0cb9. No samples were removed.

9.1 Research questions

- **RQ1 - Exact-domain correctness.** Does the integrated policy compiler produce only accepted passwords, deterministic replays, and zero same-lineage duplicates across large generation runs?
- **RQ2 - Authorization confinement.** Does exact request binding prevent one capability from authorizing other contexts/generations, and how much spill is produced by projected/wildcard negative controls?
- **RQ3 - Endpoint-compromise blast radius.** What secret types and output authority are present before, during, and after one authorized derivation?
- **RQ4 - Root-Epoch healing.** After complete disclosure of the old root, which credentials remain derivable before, during, and after migration to an independently generated new epoch?
- **RQ5 - Failure safety.** Do crash, timeout, lost-response, delayed-replication, and contradictory-evidence injections preserve the migration invariants?
- **RQ6 - Threshold feasibility.** Does an actively secure honest-majority MPC execution agree with an independent exact-domain fixed vector, and what loopback cost does the generic circuit impose?
- **RQ7 - Scalability.** How do policy-space size, password-history window, number of managed credentials, and capability-verification state affect compile time, derivation latency, and migration cost?

9.2 Executable protocol evidence

The Python semantic model executes nine protocol tests, while the Rust research implementation adds canonical binary request encoding, Ed25519 capabilities, a durable SQLite replay ledger, and a descriptor-only SQLite migration journal. The complete Rust suite contains 99 passing tests, including nine ASTER-specific tests. These are protocol and implementation checks rather than evidence for threshold secrecy.

Table 2. Semantic reference-model tests.

Test	Property exercised	Result
Exact-scope capability cannot amplify	Service/generation binding and replay budget	Pass
Same-epoch sequence injectivity	5,000 generations, no duplicates in tested domain prefix	Pass
Deterministic reconstruction	Same state returns same password	Pass
Root-Epoch replacement changes credential family	Independent epoch key separation	Pass
Cross-epoch history exclusion	Candidate excluded from tested old-history set	Pass
Unknown outcome preserves both epochs	No premature commit/retirement	Pass
Commit enables safe old-epoch retirement	Dependency guard	Pass
Unknown state blocks retirement	Safe-retirement invariant	Pass
Old root does not instantiate new epoch	Post-compromise separation in semantic model	Pass

The semantic model deliberately exposes no performance claim. Its local permutation stand-in is used only to make state transitions and negative controls executable.

9.3 RQ1: exact policy-space correctness

The experiment processed a public corpus of 270 policy records. The provenance filter classified 121 policies as exactly translated. For every exact translation P, the implementation:

1. compiled P under a fixed state/memory/time budget;
2. verified Rank(Unrank(r))=r for random and boundary ranks;
3. derived $min(100000, N_P)$ sequential generations in each tested lineage;
4. checked policy membership, deterministic replay, and duplicates;
5. recorded N_P, H_{eff}, automaton states, compiler time, RSS, and online Rank/Unrank latency.

All 121 exact translations compiled. The configured permutation implementation completed full 100,000-generation sequences for 97 policies and failed closed for 24 accepted domains above its 512-bit implementation ceiling. Across 9,700,000 derived credentials, the harness found zero policy violations, same-lineage duplicates, deterministic-replay mismatches, or Rank/Unrank inverse failures. Median policy compilation time was 80 ms, P95 was 1,812 ms, and the maximum was 59,439 ms. The largest compiled automaton had 49,401 states; the largest exact domain was 840.96 bits. These figures establish the tested implementation boundary, not universal support for all machine-readable policies.

Table 3. Exact-policy-space results.

Metric	Result
Source policy records	270
Exactly translated / compiled	121 / 121
Full permutation sequences / fail closed	97 / 24
Generated credential instances	9,700,000
Policy violations	0
Same-lineage duplicates	0
Replay / Rank-Unrank failures	0 / 0
Median / P95 compile time	80 / 1,812 ms
Maximum automaton states	49,401
Largest exact domain	840.96 bits

The acceptance criterion was zero violations, duplicates, and replay mismatches for every successful derivation; the measured run met that criterion. Domains outside the configured implementation budget were reported rather than approximated.

9.4 RQ2: authorization confinement and negative controls

The harness constructed 32 contexts spanning service, account, lineage, policy epoch/hash, Root-Epoch, and generation. It evaluated three capability modes as protocol configurations, not as competing published systems:

- **Exact**: binds every ASTER field;
- **Projected**: intentionally omits selected fields, e.g. generation and lineage;
- **Wildcard**: authorizes all contexts for a test vault.

For each authorization budget $q \in \{1, 2, 4, 8, 16, 32\}$, the harness attempted every candidate request and counted distinct accepted outputs.

Exact capabilities produced zero unauthorized spill for every q. At $q=1$, projecting the capability to service/account accepted eight contexts and spilled seven; a vault wildcard accepted all 32 and spilled 31. At $q=4$, both broad modes accepted all 32 contexts and spilled 28. Each of eight single-check ablations produced a stored witness: expiry, revocation, durable nonce/use accounting, freshness generation, Root-Epoch, password generation, lineage, and the policy-hash/epoch pair. This shows that threshold key secrecy alone does not constrain a broadly authorized interface.

Table 4. Authorization spill under controlled scope policies.

Capability binding	Candidate contexts	Intended outputs	Accepted outputs	Unauthorized spill
Complete ASTER context	32	1	1	0
Service/account only	32	1	8	7
Wildcard negative control	32	1	32	31

The exact configuration satisfied Theorem 5 in the bounded universe. The projected and wildcard configurations violate the one-capability/one-output contract by construction and serve only as negative controls.

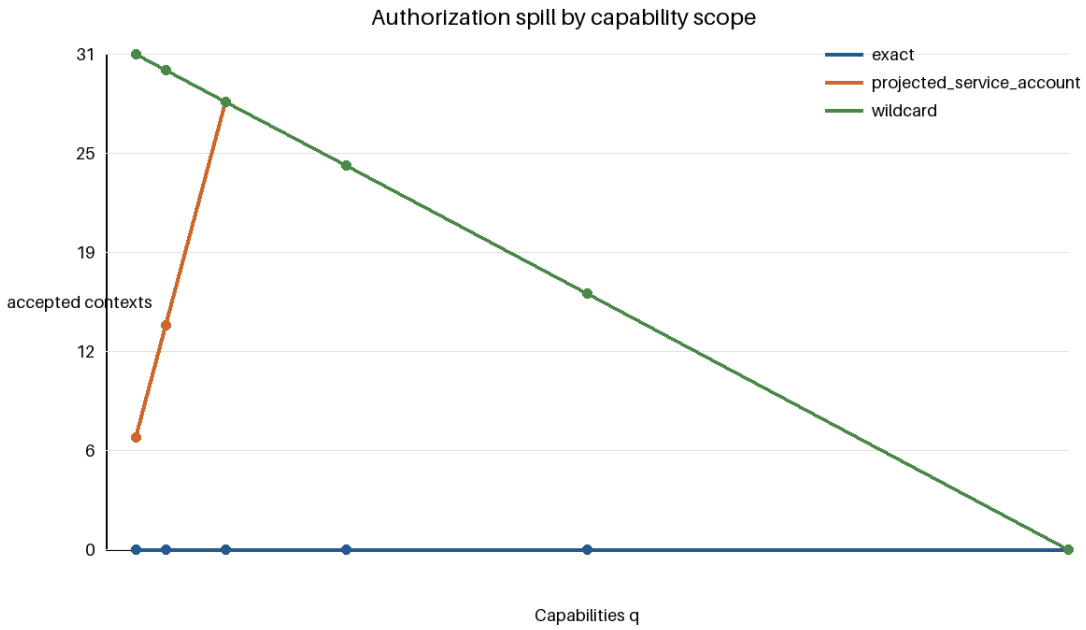

Figure 1. Authorization spill in the 32-context experiment. Exact binding remains at zero; broader bindings admit the contexts their omitted fields leave unconstrained.

9.5 RQ3: endpoint-compromise blast radius

The endpoint-compromise experiment distinguishes **observed plaintext** from **derivation authority**. A research-only typed inventory records whether a Root-Epoch key, reusable lineage key, capability, or plaintext password crosses the endpoint API before authorization, while one credential is returned, and after use. A controlled attacker harness then attempts all 32 contexts without requesting new approval.

The inventory and attacker harness measured:

- passwords directly present in the snapshot;
- additional outputs derivable offline;
- additional outputs obtainable by replaying observed tokens;
- additional outputs obtainable without new Approval Authority interaction;
- additional outputs obtainable if the Approval Authority is also compromised.

For ASTER's exact mode, the unauthorized-output count without new approval was 0 before the request, 1 during the authorized output window, and 0 after use. The API inventory reported no Root-Epoch or reusable lineage key at any capture time. When the Approval Authority was deliberately marked compromised, the harness could authorize all 32 contexts; this is the expected boundary described in Section 7.4. Separate ablation fixtures confirm that deliberately placing a reusable root, a reusable lineage key, or a broad remote capability at the endpoint expands authority, but these fixtures are attack configurations rather than experimental comparisons with unpublished systems. The inventory is structural/API evidence, not whole-process memory forensics or a proof of zeroization.

9.6 RQ4: Root-Epoch healing

The experiment created $M = 100$ credential records committed to epoch e, exposed the complete old Root-Epoch key to the attacker harness, generated independent epoch $e + 1$, and migrated records in batches.

After each batch, the harness tested every currently committed descriptor against the attacker's retained old-root state.

An additional negative control refreshed shares without changing K_e.

At migration counts 0, 10, 25, 50, 75, and 100, the numbers still derivable from the old root were exactly 100, 90, 75, 50, 25, and 0. No conclusively migrated credential remained derivable from old-root state alone. Share refresh preserved all sampled outputs, correctly demonstrating non-healing. UnknownOutcome records remain classified as not safely healed until evidence resolves the authoritative committed descriptor. Candidate selection for history windows $h = 0, 1, 5, 10, 24$ took 712, 1,305, 2,344, 5,928, and 14,102 microseconds in the process-local semantic backend; these values characterize local protocol work only and are not MPC timings.

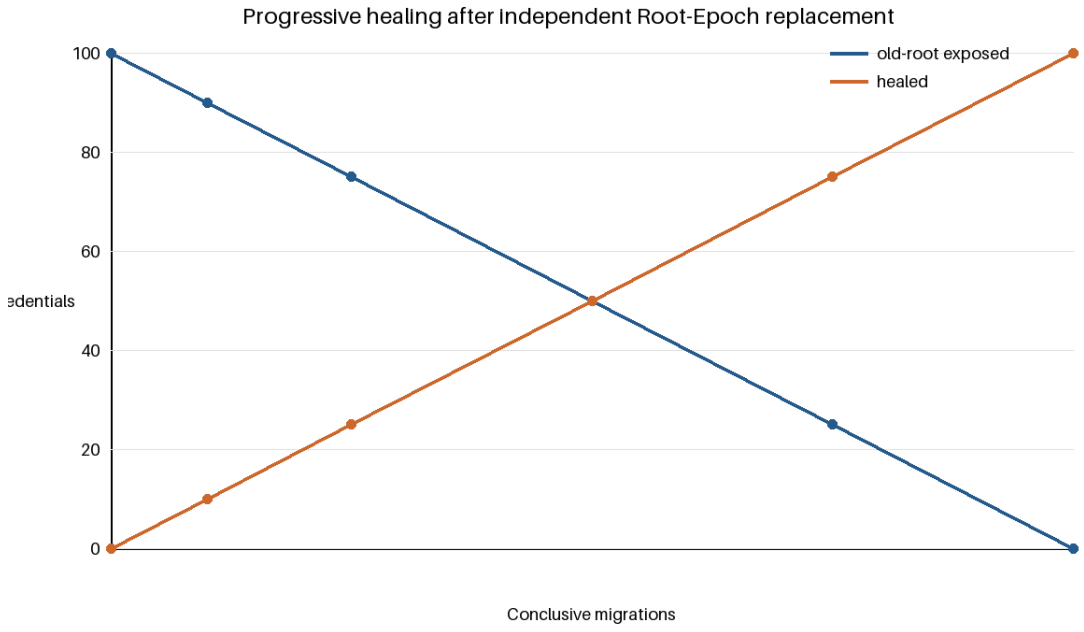

Figure 2. Credentials derivable from the compromised old root as completed Root-Epoch migrations increase. The share-refresh negative control remains horizontal at 100.

9.7 RQ5: failure injection and model checking

The migration harness injected faults at persistence and network boundaries:

- crash before candidate journal fsync;
- crash after fsync but before submission;
- request dropped before target commit;
- target commits but response is dropped;
- delayed LDAP replication;
- new-password verification unavailable;
- old-password verification unavailable;
- both passwords accepted;
- neither password accepted;
- contradictory evidence sources;
- process restart during UnknownOutcome;
- stale local snapshot replay;
- stale capability replay;
- stale Root-Epoch/freshness generation.

The required invariants are:

- **I1:** local committed state never advances solely from generic transport success;
- **I2:** UnknownOutcome preserves enough authenticated metadata to reconstruct both old and candidate passwords while their epochs remain available;
- **I3:** an epoch referenced by committed/pending/history state cannot be retired;
- **I4:** exact capabilities cannot be replayed outside their scope or use budget;
- **I5:** commit to the new epoch requires the adapter's configured singleton evidence condition.

A TLA+ model includes a positive configuration and eight deliberately broken configurations: commit on HTTP success, drop the candidate on timeout, ignore replay, expiry, freshness, Root-Epoch, or generation binding, and retire a referenced old epoch.

The adapter matrix ran 16 scenarios against two local targets with three repetitions, producing 96 traces. The HTTP target is a real loopback service. The second target is an independent TCP process with durable verifier hashes and modeled delayed authoritative readback; it is LDAP-style, not OpenLDAP or a replicated directory cluster. The harness observed zero commit-invariant violations, zero uncertainty-preservation violations, and zero password columns in the journal schema. TLC generated 2,426 states, found 777 distinct states at depth 11, exhausted the bounded positive state space with no invariant violation, and produced the intended counterexample for every negative configuration. The checked model is an abstraction, not a cryptographic or implementation proof.

Table 5. Fault-injection and model-checking results.

Evidence	Result
Adapter scenarios / traces	16 / 96
Commit / uncertainty invariant violations	0 / 0
Positive TLC generated / distinct states	2,426 / 777
Positive maximum depth	11
Negative controls with counterexample	8 / 8

9.8 RQ6: threshold/MPC feasibility and cost

The cryptographic experiment executed the fixed circuit of Section 8.4 with MP-SPDZ's malicious honest-majority Shamir/BMR backend. Three parties used corruption threshold $t=1$ and five parties used $t=2$; every configured party participated online. Each configuration ran the same public request, generation 42, and exact domain $N=1,000,003$ three times in a single-host loopback Docker environment. The independent OpenSSL-based reference returned rank 70,397 for the three-party private inputs and rank 697,614 for the five-party inputs. Every MPC execution opened the same success bit and rank as its reference.

Table 6. Malicious honest-majority MP-SPDZ fixed-vector measurements.

Parties	Corruption threshold	Samples	Median / P95 / P99 (s)	Runtime rounds	MB per party	Global MB	Fixed-vector agreement
3	1	3	33.011 / 35.755 / 35.999	22,800	790.705	2,372.13	Yes
5	2	3	125.534 / 127.092 / 127.230	35,104	2,445.15	12,225.8	Yes

These measurements establish functional agreement and a concrete cost boundary, not interactive production performance. Even on loopback, the generic bit-level circuit required tens to hundreds of seconds and hundreds to thousands of megabytes per party. The five-party configuration increased both communication and runtime substantially. The experiment did not measure LAN/WAN behavior, DKG, share refresh, or history-window MPC cost, and no such figures are inferred. A deployment-oriented implementation therefore requires a purpose-built threshold exact-domain permutation or a materially lower-round secure-computation realization; the present circuit is retained as reproducible feasibility evidence.

9.9 RQ7: scalability

The scalability experiment paired one public credential-metadata row with one durable capability-use row and scaled the SQLite ledger from 10^2 to 10^5 records. This isolates public indexing/replay state; it does not include password values or MPC computation.

At 100, 1,000, 10,000, and 100,000 paired rows, database sizes were 36,864; 221,184; 2,015,232; and 20,168,704 bytes. Median indexed lookup latency was 18.177, 18.513, 20.165, and 29.646 microseconds; corresponding P95 values were 21.646, 20.916, 26.080, and 33.231 microseconds. The 100,000-row insertion took 1,653.35 ms. RQ1 separately identified the exact-policy implementation's fail-closed boundary: 24 exact domains exceeded the configured 512-bit permutation limit.

Table 7. Durable public-state scalability.

Rows	Database bytes	Insert ms	Lookup median / P95 / P99 (microseconds)
100	36,864	1.95	18.177 / 21.646 / 85.847
1,000	221,184	16.05	18.513 / 20.916 / 32.749
10,000	2,015,232	158.95	20.165 / 26.080 / 58.173
100,000	20,168,704	1,653.35	29.646 / 33.231 / 63.580

9.10 Reproducibility requirements

The artifact includes:

- fixed dependency lockfiles;
- build scripts for all evaluator parties;
- public policy corpus translation provenance;
- deterministic experiment configuration files;
- raw JSON/CSV output;
- scripts that regenerate every table and figure;
- TLA+ model and all positive/negative configurations;
- MP-SPDZ-emitted communication accounting;
- fault-injection harness;
- exact software/hardware environment capture;
- a quick and full top-level reproduction target.

Measured results must be generated from raw files, not manually copied into source tables.

10. Discussion and Limitations

10.1 ASTER remains a legacy compatibility mechanism

ASTER should not be deployed where a relying party can adopt passkeys, WebAuthn, client certificates, or another phishing-resistant public-key mechanism. Its purpose is to reduce credential-management risk during migration, not to preserve password authentication indefinitely.

10.2 Plaintext necessarily exists at the legacy boundary

A password-only target ultimately receives a plaintext-equivalent password. If the endpoint is already compromised during legitimate use, malware can capture that password. ASTER reduces the scope of reusable secret material on the endpoint but cannot remove the target’s protocol requirement.

10.3 Threshold deployment independence is operational, not mathematical

A threshold deployment is not secure if one administrative credential, cloud control plane, hypervisor, or compromised endpoint controls more evaluator domains than the assumed corruption threshold permits. The reported MP-SPDZ configurations tolerate one corrupted party among three or two among five under the backend’s stated honest-majority model, but all configured parties participate online. Production assurance depends on genuinely independent compromise domains, not node count alone.

10.4 Approval can become the new concentration point

Exact-scope capabilities deliberately move authority from a reusable derivation key to an approval decision. If the Approval Authority is fully automated and directly callable by the compromised endpoint without independent policy, the endpoint may obtain a stream of individually valid outputs. This still differs from offline root compromise but may be operationally unacceptable. High-value deployments should use an independent approval domain, device/user-presence controls, rate limits, and differentiated policy for migration.

10.5 Root-Epoch healing is gradual

After K_e disclosure, credentials remain exposed until each relying party actually accepts a new-epoch password. ASTER therefore provides **progressive healing**. The migration exposure curve is an important operational metric: it identifies exactly which records remain derivable by the old-root attacker.

10.6 Password-history dependence delays old-key erasure

If a target requires a history window that reaches back into epoch e, ASTER may need to retain enough old-epoch evaluator material to rederive those historical passwords during migration. This creates a tension between immediate key erasure and strict history exclusion. Possible mitigations include migrating the whole required history window before retirement, using a target-admin history reset, or storing a privacy-preserving history commitment whose security properties must be analyzed separately. ASTER does not hide this trade-off.

10.7 Side channels and implementation leakage

Generic MPC and FPE implementations may leak through timing, memory access, logs, crash dumps, or network metadata. The paper's main proofs are protocol-level and do not establish constant-time behavior. A production deployment requires independent cryptographic review, secret-zeroization review, hardened logging, and host isolation.

10.8 Policy translation is trusted input

The exact compiler can prove properties only of the machine-readable policy it receives. If a target's undocumented semantics differ from that policy, the generated password may still be rejected. Translation provenance and fail-closed handling are therefore part of the reproducibility artifact.

10.9 Backend and evaluation boundary

The fully adaptive threshold POPRF construction with proactive key refresh [9] reinforces the need to avoid claiming distributed evaluation or share refresh as novel. ASTER's MP-SPDZ circuit composes established components and is intentionally not presented as a new threshold primitive. Its three repetitions per configuration, single-host loopback topology, fixed 20-bit superset domain, and fixed cycle-walk cap are sufficient for an auditable feasibility result but not for deployment sizing. The HTTP adapter is a real loopback service; the second adapter is an independent durable LDAP-style process, not OpenLDAP or a replicated directory cluster. The secret-inventory and journal-schema checks likewise do not prove erasure from process memory, swap, or crash dumps.

10.10 Scope of the contribution

ASTER's contribution is the protocol relation among exact accepted-policy-space sequencing, threshold evaluation without endpoint key reconstruction, exact per-generation authorization, independent root replacement after complete old-root compromise, cross-epoch history exclusion, and failure-safe ambiguous remote password rotation. Each underlying primitive has prior art and is cited separately. The security claims therefore apply to the composed credential lifecycle and its stated assumptions, not to novelty of the component primitives.

11. Conclusion

This paper introduced ASTER, a threshold credential-derivation architecture for legacy password systems that separates three questions often collapsed into one: how a password is selected from the target's exact accepted domain, who is authorized to obtain a specific credential output, and how the system heals after the old root is already compromised.

ASTER compiles a bounded password policy into an exact finite language and maps password generations through a context-separated permutation, yielding deterministic policy compliance and strict same-lineage non-repetition. The normal endpoint does not receive the Root-Epoch key or a reusable lineage key; instead, each output requires a short-lived capability bound to the complete credential context and generation. This converts endpoint compromise from implicit possession of a large derivation capability into an explicitly budgeted output authority, subject to the independence of the Approval Authority and evaluator domains.

The second contribution is Root-Epoch healing. Proactive share refresh of an unchanged key is valuable but cannot revoke an attacker who already knows that key. ASTER instead generates an independent Root-Epoch and migrates relying-party passwords individually. Cross-epoch history exclusion occurs inside the distributed computation, and ambiguous remote password changes retain both deterministic reconstruction paths until evidence is conclusive. Old epochs are erased only after no credential, pending operation, or required history depends on them.

The artifact separates semantic, cryptographic, and systems evidence. Exact-policy experiments covered 9.7 million derivations; capability experiments exposed the amplification caused by omitted scope fields; independent Root-Epoch replacement produced progressive healing while share refresh did not; fault injection and bounded model checking preserved the stated lifecycle invariants; and malicious honest-majority MP-SPDZ executions matched independent fixed vectors while revealing the high cost of the generic circuit. These results support ASTER as a credential-protocol contribution, not a new cryptographic primitive or a production-ready deployment claim.

Declarations

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Conflict of interest

The author declares no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Data and code availability

The accompanying artifact includes source code, fixed test vectors, experiment configurations, raw JSON/JSONL results, generated tables and figures, TLA+ models, dependency versions, and quick/full reproduction scripts. Repository-identifying information can be anonymized during double-blind review where required.

Use of generative AI and AI-assisted technologies

During manuscript preparation, the author used AI-assisted coding and language tools to help develop executable reference models, organize experiments, inspect consistency, and edit prose. The author remains responsible for the research design, implementation, evidence, citations, interpretation, and final manuscript.

References

[1] B. Ross, C. Jackson, N. Miyake, D. Boneh, J. C. Mitchell, Stronger Password Authentication Using Browser Extensions, in: 14th USENIX Security Symposium, USENIX Association, 2005.

[2] M. Horsch, A. T. Hülsing, J. Buchmann, PALPAS - PAssword Less PAssword Synchronization, in: 10th International Conference on Availability, Reliability and Security (ARES), IEEE, 2015, pp. 30-39. https://doi.org/10.1109/ARES.2015.23.

[3] F. Al Maqbali, C. J. Mitchell, AutoPass: An Automatic Password Generator, in: 2017 International Carnahan Conference on Security Technology (ICCST), IEEE, 2017, pp. 1-6. https://doi.org/10.1109/CCST.2017.8167791.

[4] National Institute of Standards and Technology, Multi-Party Threshold Cryptography Project, NIST Computer Security Resource Center, accessed August 2026.

[5] L. T. A. N. Brandão, R. Peralta, NIST IR 8214C: NIST First Call for Multi-Party Threshold Schemes, National Institute of Standards and Technology, January 2026. https://doi.org/10.6028/NIST.IR.8214C.

[6] A. Everspaugh, R. Chaterjee, S. Scott, A. Juels, T. Ristenpart, The Pythia PRF Service, in: 24th USENIX Security Symposium, USENIX Association, 2015, pp. 547-562.

[7] M. Geihs, H. Montgomery, LaKey: Efficient Lattice-Based Distributed PRFs Enable Scalable Distributed Key Management, in: 33rd USENIX Security Symposium, USENIX Association, 2024, pp. 4319-4335.

[8] A. Davidson, A. Faz-Hernandez, N. Sullivan, C. A. Wood, Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups, RFC 9497, IETF, 2023. https://doi.org/10.17487/RFC9497.

[9] R. Baecker, P. Gerhart, D. Rausch, D. Schröder, A Fully-Adaptive Threshold Partially-Oblivious PRF, in: Advances in Cryptology - CRYPTO 2025, LNCS 16005, Springer, 2025, pp. 569-597. https://doi.org/10.1007/978-3-032-01901-1_18.

[10] A. Gautam, S. Lalani, S. Ruoti, Improving Password Generation Through the Design of a Password Composition Policy Description Language, in: Eighteenth Symposium on Usable Privacy and Security (SOUPS 2022), USENIX Association, 2022, pp. 541-560.

[11] M. Grilo, J. Campos, J. F. Ferreira, J. B. Almeida, A. Mendes, Verified Password Generation from Password Composition Policies, in: Integrated Formal Methods (iFM 2022), LNCS 13274, Springer, 2022, pp. 271-288. https://doi.org/10.1007/978-3-031-07727-2_15.

[12] J. Black, P. Rogaway, Ciphers with Arbitrary Finite Domains, in: Topics in Cryptology - CT-RSA 2002, LNCS 2271, Springer, 2002, pp. 114-130. https://doi.org/10.1007/3-540-45760-7_9.

[13] M. Bellare, T. Ristenpart, P. Rogaway, T. Stegers, Format-Preserving Encryption, in: Selected Areas in Cryptography 2009, Springer, 2009, pp. 295-312. https://doi.org/10.1007/978-3-642-05445-7_19.

[14] M. Dworkin, N. Mouha, NIST SP 800-38G Rev. 1 (Second Public Draft): Recommendation for Block Cipher Modes of Operation: Methods for Format-Preserving Encryption, National Institute of Standards and Technology, February 2025. https://doi.org/10.6028/NIST.SP.800-38Gr1.2pd.

[15] A. Shamir, How to Share a Secret, Communications of the ACM 22(11) (1979) 612-613. https://doi.org/10.1145/359168.359176.

[16] V. Nair, D. Song, Multi-Factor Key Derivation Function (MFKDF) for Fast, Flexible, Secure, & Practical Key Management, in: 32nd USENIX Security Symposium, USENIX Association, 2023, pp. 2097-2114.

[17] M. Scarlata, M. Backendal, M. Haller, MFKDF: Multiple Factors Knocked Down Flat, in: 33rd USENIX Security Symposium, USENIX Association, 2024, pp. 4301-4318.

[18] M. Keller, MP-SPDZ: A Versatile Framework for Multi-Party Computation, in: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2020, pp. 1575-1590. https://doi.org/10.1145/3372297.3417872.